Understanding the Flyweight Design Pattern

A Case Study Approach

Md Mehedi Hasan Maruf, Roll: 2003037
Md Shahidul Islam Dipu, Roll: 2003040
Eazdan Mostafa Rafin, Roll: 2003055

January 7, 2025

What is the Flyweight Pattern?

- The Flyweight pattern helps save memory by sharing common data between similar objects.
- Objects are split into:
  - **Shared Data (Intrinsic):** Data that doesn't change and is reused.
  - **Unique Data (Extrinsic):** Specific data provided by the user when needed.

Key Concepts

- **Flyweight:** The shared object containing reusable data.
- **Flyweight Factory:** Manages and ensures Flyweights are shared properly.
- **Client:** Passes unique data and uses Flyweights for shared data.

Case Study: Enemy Objects in a Game

Problem Statement

Efficiently manage numerous in-game enemies by addressing:

- High memory usage from duplicated shared attributes.
- Performance issues in resource-constrained systems.
- Challenges in handling shared and unique attributes effectively.

Requirements

- Optimize memory by reusing shared attributes (Flyweight pattern).
- Support unique attributes like position and health.
- Allow scalable creation of multiple enemies efficiently.

Flyweight Structure

Component	Description	Example in Code
Flyweight	Shared object for intrinsic state	`EnemyFlyweight`
Flyweight Factory	Manages Flyweight creation and reuse	`EnemyFactory`
Client	Supplies extrinsic state for each instance	`Enemy`
Extrinsic State Provider	Supplies unique data like position and health	`main_game_loop()`

Code: Flyweight Class

```python
class EnemyFlyweight:
    """
    Flyweight class that holds shared data (intrinsic state).
    """
    def __init__(self, enemy_type, health, attack, defense, texture):
        self.enemy_type = enemy_type
        self.health = health
        self.attack = attack
        self.defense = defense
        self.texture = texture

    def display_shared_properties(self):
        """
        Display the shared properties of the enemy type.
        """
        print("Shared Properties:")
        print(f"  Enemy Type: {self.enemy_type}")
        print(f"  Health: {self.health}")
        print(f"  Attack: {self.attack}")
        print(f"  Defense: {self.defense}")
        print(f"  Texture: {self.texture}")
        print("-" * 30)
```

Code: Factory Class

```
1  class EnemyFactory:
2      """
3      Factory class to manage and provide Flyweight instances.
4      """
5      def __init__(self):
6          self._enemies = {}
7
8      def get_enemy(self, enemy_type, health, attack, defense, texture):
9          """
10         Get a shared Flyweight instance or create one if it doesn't exist.
11         """
12         key = (enemy_type, health, attack, defense, texture)
13         if key not in self._enemies:
14             print(f"Creating new Flyweight for {enemy_type}")
15             self._enemies[key] = EnemyFlyweight(enemy_type, health, attack, defense, texture)
16         return self._enemies[key]
```

Code: Enemy Class

```python
class Enemy:
    """
    Interface for an enemy instance that combines Flyweight (shared data)
    and unique data.
    """
    def __init__(self, flyweight, position, current_health):
        self.flyweight = flyweight  # Shared data
        self.position = position  # Extrinsic data
        self.current_health = current_health  # Extrinsic data

    def display_unique_properties(self):
        """
        Display the unique properties of the enemy instance.
        """
        print("Unique Properties:")
        print(f"  Position: {self.position}")
        print(f"  Current Health: {self.current_health}")
        print("-" * 30)

    def render(self):
        """
        Render the enemy, showing both shared and unique properties.
        """
        self.flyweight.display_shared_properties()
        self.display_unique_properties()
```

Code: Main Game Loop

```
1  def main_game_loop():
2      """
3      Interface for the main game loop that handles enemy creation and rendering.
4      """
5      factory = EnemyFactory()
6
7      # Example enemies
8      orc = factory.get_enemy("Orc", 100, 15, 5, "orc_texture.png")
9      goblin = factory.get_enemy("Goblin", 50, 10, 2, "goblin_texture.png")
10
11     # Create enemy instances with unique data
12     enemies = [
13         Enemy(orc, (10, 20), 80),
14         Enemy(orc, (15, 25), 60),
15         Enemy(goblin, (5, 10), 40),
16         Enemy(goblin, (7, 12), 30),
17     ]
18
19     # Render enemies
20     for enemy in enemies:
21         enemy.render()
```

Log Output (Part 1)

```
Creating new Flyweight for Orc
Creating new Flyweight for Goblin

Shared Properties:
  Enemy Type: Orc
  Health: 100
  Attack: 15
  Defense: 5
  Texture: orc_texture.png
------------------------------
Unique Properties:
  Position: (10, 20)
  Current Health: 80
------------------------------
Shared Properties:
  Enemy Type: Orc
  Health: 100
  Attack: 15
  Defense: 5
  Texture: orc_texture.png
------------------------------
Unique Properties:
  Position: (15, 25)
  Current Health: 60
```

Log Output (Part 2)

```
Shared Properties:
  Enemy Type: Goblin
  Health: 50
  Attack: 10
  Defense: 2
  Texture: goblin_texture.png
------------------------------
Unique Properties:
  Position: (5, 10)
  Current Health: 40
------------------------------
Shared Properties:
  Enemy Type: Goblin
  Health: 50
  Attack: 10
  Defense: 2
  Texture: goblin_texture.png
------------------------------
Unique Properties:
  Position: (7, 12)
  Current Health: 30
------------------------------
```

Real-World Analogy

- Think about a chess game:
  - Shared data: What type of piece it is (e.g., Pawn, King).
  - Unique data: Where the piece is on the board.
  - You don't need a separate object for each piece on every square—just reuse the shared piece types!

Pros and Cons of Flyweight Pattern

Pros:

- Saves a lot of memory by reusing objects.
- Perfect for applications with many similar objects.

Cons:

- Can make your code harder to understand and maintain.
- Only works well when shared data is immutable.

Summary and Q&A

- The Flyweight pattern is a great way to save memory by reusing common data.
- It's best suited for situations with many similar objects, like text editors or games.
- Keep in mind that it can add complexity to your code!